



ТЕХНИЧЕСКИ УНИВЕРСИТЕТ - СОФИЯ

Катедра „Компютърни системи“

КУРСОВ ПРОЕКТ
ПО БАЗИ ДАННИ

Студент: Емилиян Георгиев Николов

Факултетен номер: 121224184 Група: 42^a

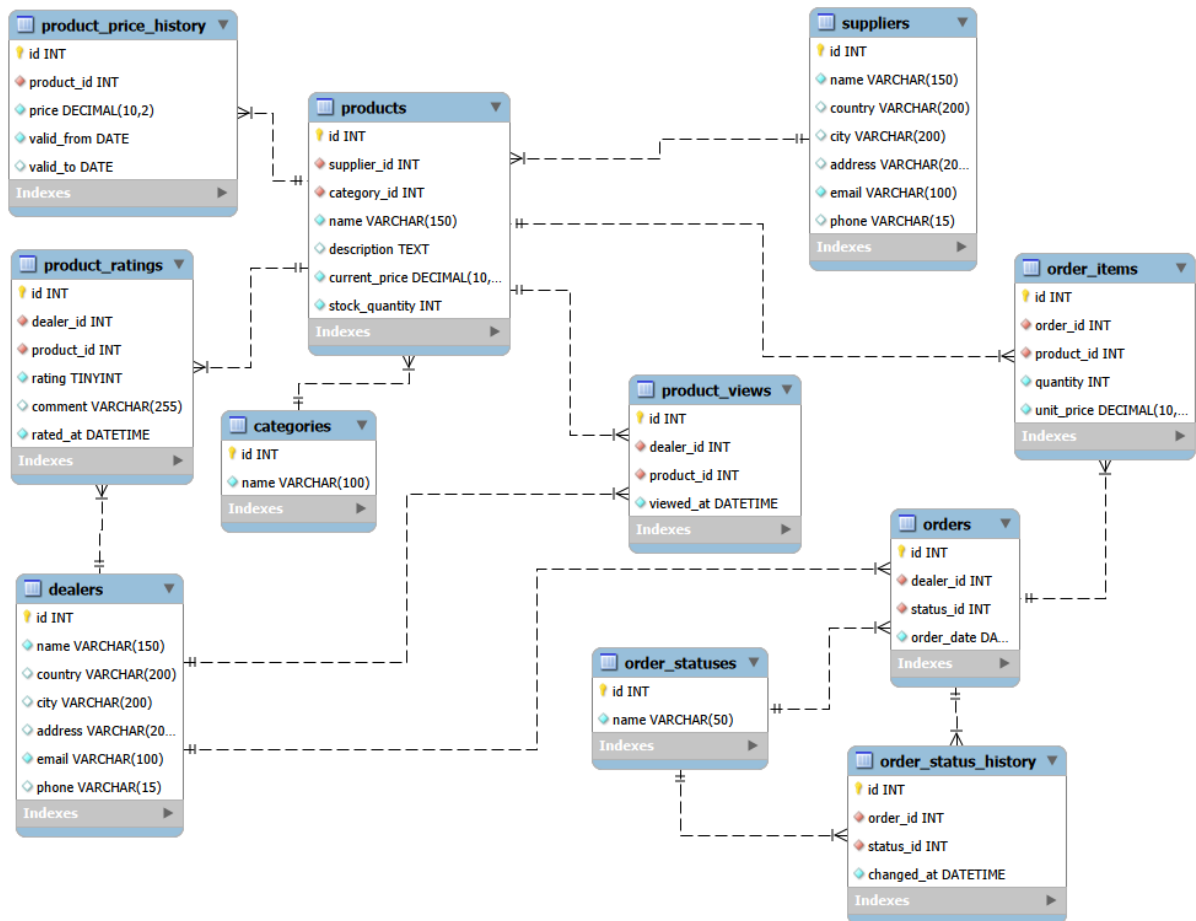
Тема №4

Разработете база данни за система за поддържане на доставчици, стоки и цени, които ще се разглеждат от различни клиенти - „дилъри“. Да се пази история кой клиент - коя стока е разгледал и кой клиент - какви стоки е поръчал. Да се направи и рейтинг на стоките. Да може да се извеждат различни статистически данни - напр. най-продавани артикули за конкретен период, промяна в цената на дадена стока на годишна база и т.н.

1. Да се проектира база от данни и да се представи ER диаграма с подходящите заявки за средата MySQL.
2. Напишете заявка, в която демонстрирате SELECT с ограничаващо условие по избор.
3. Напишете заявка, в която използвате агрегатна функция и GROUP BY по ваш избор.
4. Напишете заявка, в която демонстрирате INNER JOIN по ваш избор.
5. Напишете заявка, в която демонстрирате OUTER JOIN по ваш избор.
6. Напишете заявка, в която демонстрирате вложен SELECT по ваш избор.
7. Напишете заявка, в която демонстрирате едновременно JOIN и агрегатна функция.
8. Създайте тригер по ваш избор.
9. Създайте процедура, в която демонстрирате използване на курсор.

1. Проектиране на базата данни, представяне на ER диаграма и създаване на таблици и добавяне на данни в тях.

Основните обекти, за които трябва да се съхранява информация според заданието, са: доставчици (suppliers), продукти (products) и клиенти - „дилъри“ (dealers). Освен тях, в базата данни са създадени и допълнителни таблици, които разширяват функционалността на системата. Една от тези таблици е categories, която служи за класифициране на продуктите в различни категории. Това улеснява тяхното организиране, търсене и анализ. Друга важна таблица е product_price_history, която съхранява история на цените на продуктите. В нея се пази информация за това кой продукт каква цена е имал, от коя дата е валидна и до коя дата (ако има край на валидността). Това позволява проследяване на промените в цените във времето. За да се следи поведението на клиентите, е създадена таблицата product_views, в която се записва кой дилър коя стока е разгледал и кога. Това дава възможност за анализ на интереса към различни продукти. Допълнително, чрез таблицата product_ratings се реализира рейтинг система за стоките. В нея се съхранява оценката, която даден дилър е поставил на продукт, както и евентуален коментар и дата на оценяване. За обработка на поръчки са създадени таблиците orders и order_items. Таблицата orders съдържа основна информация за всяка поръчка - кой дилър я е направил, нейния статус и дата. В order_items се съхраняват конкретните продукти в дадена поръчка, тяхното количество и цена към момента на поръчката. Също така е въведена таблицата order_statuses, която дефинира възможните състояния на поръчките (например „нова“, „обработка“, „изпратена“, „завършена“), както и order_status_history, в която се пази история на промените в статуса на всяка поръчка. За проектирането на базата ще използваме модела ER-диаграма (Entity Relationship Diagram):



Заявките, с които създаваме базата данни и таблиците са:

```

DROP DATABASE IF EXISTS dealer_system;
CREATE DATABASE dealer_system;
USE dealer_system;

```

```

CREATE TABLE suppliers(
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(150) NOT NULL,
    country VARCHAR(200),
    city VARCHAR(200),
    address VARCHAR(200),
    email VARCHAR(100) NOT NULL UNIQUE,
    phone VARCHAR(15)
);

```

```

CREATE TABLE categories (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE
);

```

```

CREATE TABLE products(

```

```

    id INT AUTO_INCREMENT PRIMARY KEY,
    supplier_id INT NOT NULL,
    category_id INT NOT NULL,
    name VARCHAR(150) NOT NULL,
    description text,
    current_price DECIMAL(10, 2) NOT NULL,
    stock_quantity INT NOT NULL DEFAULT 0,

    FOREIGN KEY (supplier_id)
        REFERENCES suppliers(id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE,
    FOREIGN KEY (category_id)
        REFERENCES categories(id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE,

    CHECK (current_price > 0),
    CHECK (stock_quantity >= 0)
);

CREATE TABLE product_price_history(
    id INT AUTO_INCREMENT PRIMARY KEY,
    product_id INT NOT NULL,
    price DECIMAL(10, 2) NOT NULL,
    valid_from DATE NOT NULL,
    valid_to DATE,

    FOREIGN KEY(product_id)
        REFERENCES products(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,

    CHECK (price > 0),
    CHECK (valid_to IS NULL OR valid_to >=
valid_from)
);

CREATE TABLE dealers(
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(150) NOT NULL,
    country VARCHAR(200),
    city VARCHAR(200),
    address VARCHAR(200),

```

```
    email VARCHAR(100) NOT NULL UNIQUE,  
    phone VARCHAR(15)  
);  
  
CREATE TABLE product_views(  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    dealer_id INT NOT NULL,  
    product_id INT NOT NULL,  
    viewed_at DATETIME NOT NULL DEFAULT  
CURRENT_TIMESTAMP,  
  
    FOREIGN KEY(dealer_id)  
        REFERENCES dealers(id)  
        ON DELETE RESTRICT  
        ON UPDATE CASCADE,  
    FOREIGN KEY(product_id)  
        REFERENCES products(id)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE  
);  
  
CREATE TABLE product_ratings(  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    dealer_id INT NOT NULL,  
    product_id INT NOT NULL,  
    rating TINYINT NOT NULL,  
    comment VARCHAR(255),  
    rated_at DATETIME NOT NULL DEFAULT  
CURRENT_TIMESTAMP,  
  
    UNIQUE(dealer_id, product_id),  
  
    FOREIGN KEY(dealer_id)  
        REFERENCES dealers(id)  
        ON DELETE RESTRICT  
        ON UPDATE CASCADE,  
    FOREIGN KEY(product_id)  
        REFERENCES products(id)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE,  
  
    CHECK (rating BETWEEN 1 AND 5)  
);
```

```

CREATE TABLE order_statuses (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE orders(
    id INT AUTO_INCREMENT PRIMARY KEY,
    dealer_id INT NOT NULL,
    status_id INT NOT NULL,
    order_date DATE NOT NULL DEFAULT (CURRENT_DATE),

    FOREIGN KEY(dealer_id)
        REFERENCES dealers(id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE,
    FOREIGN KEY(status_id)
        REFERENCES order_statuses(id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);

CREATE TABLE order_items(
    id INT AUTO_INCREMENT PRIMARY KEY,
    order_id INT NOT NULL,
    product_id INT NOT NULL,
    quantity INT NOT NULL,
    unit_price DECIMAL(10, 2) NOT NULL,

    UNIQUE(order_id, product_id),

    FOREIGN KEY(order_id)
        REFERENCES orders(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    FOREIGN KEY(product_id)
        REFERENCES products(id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE,

    CHECK (quantity > 0),
    CHECK (unit_price > 0)
);

CREATE TABLE order_status_history(

```

```

    id INT AUTO_INCREMENT PRIMARY KEY,
    order_id INT NOT NULL,
    status_id INT NOT NULL,
    changed_at DATETIME NOT NULL DEFAULT
CURRENT_TIMESTAMP,

    FOREIGN KEY(order_id)
        REFERENCES orders(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    FOREIGN KEY(status_id)
        REFERENCES order_statuses(id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);

```

Добавяме и тестови данни в таблиците:

```

INSERT INTO suppliers(name, country, city, address,
email, phone) VALUES
('Tech Solutions Ltd.', 'Germany', 'Berlin', 'Unter
den Linden 12', 'contact@techsolutions.de',
'+49301234567'),
('Global Devices Inc.', 'USA', 'San Francisco',
'Market Street 1355', 'sales@globaldevices.com',
'+14155551234'),
('ElectroWorld', 'Bulgaria', 'Sofia', 'Tsarigradsko
Shose Blvd 115', 'info@electroworld.bg',
'+35929876543');

INSERT INTO categories(name) VALUES
('Laptops'),
('Smartphones'),
('Peripherals');

INSERT INTO products(supplier_id, category_id, name,
description, current_price, stock_quantity) VALUES
(1, 1, 'Dell XPS 15', 'High-end ultrabook laptop',
3200.00, 15),
(1, 1, 'Lenovo ThinkPad X1', 'Business class laptop',
2800.00, 20),
(2, 2, 'iPhone 14 Pro', 'Apple smartphone', 2200.00,
30),
(2, 2, 'Samsung Galaxy S23', 'Android flagship
phone', 1800.00, 25),

```



```
(3, 3, 'Logitech MX Master 3', 'Wireless mouse',  
120.00, 100),  
(3, 3, 'Keychron K6', 'Mechanical keyboard', 200.00,  
60);
```

```
INSERT INTO product_price_history(product_id, price,  
valid_from, valid_to) VALUES  
(1, 3000.00, '2024-01-01', '2024-12-31'),  
(1, 3200.00, '2025-01-01', NULL),  
(3, 2000.00, '2024-06-01', '2024-12-31'),  
(3, 2200.00, '2025-01-01', NULL);
```

```
INSERT INTO dealers(name, country, city, address,  
email, phone) VALUES  
( 'Ivan Petrov Ltd.', 'Bulgaria', 'Sofia', 'Vitosha  
Blvd 25', 'ivan.petrov@dealer.bg', '+359888123456'),  
( 'Plamen Georgiev EOOD', 'Bulgaria', 'Plovdiv',  
'Kapitan Raycho 10', 'plamen.georgiev@dealer.bg',  
'+3598889654321'),  
( 'Nikola Dimitrov Trade', 'Bulgaria', 'Varna',  
'Slivnitsa Blvd 88', 'nikola.dimitrov@dealer.bg',  
'+359887112233');
```

```
INSERT INTO product_views(dealer_id, product_id)  
VALUES  
(1, 1), (1, 1), (1, 2), (1, 3),  
(2, 1), (2, 3), (2, 4),  
(3, 2), (3, 5), (3, 6);
```

```
INSERT INTO product_ratings(dealer_id, product_id,  
rating, comment) VALUES  
(1, 1, 5, 'Excellent performance'),  
(1, 3, 4, 'Very good phone'),  
(2, 1, 4, 'Solid laptop'),  
(2, 4, 5, 'Amazing device'),  
(3, 5, 3, 'Average mouse'),  
(3, 6, 5, 'Great keyboard');
```

```
INSERT INTO order_statuses(name) VALUES  
( 'New'),  
( 'Processing'),  
( 'Shipped'),  
( 'Delivered');
```

```
INSERT INTO orders(dealer_id, status_id, order_date)
VALUES
(1, 1, '2025-01-10'),
(1, 2, '2025-01-15'),
(2, 3, '2025-02-05'),
(3, 4, '2025-03-12');
```

```
INSERT INTO order_items(order_id, product_id,
quantity, unit_price) VALUES
(1, 1, 2, 3200.00),
(1, 5, 3, 120.00),
(2, 2, 1, 2800.00),
(2, 6, 2, 200.00),
(3, 3, 2, 2200.00),
(4, 4, 1, 1800.00);
```

```
INSERT INTO order_status_history(order_id, status_id,
changed_at) VALUES
(1, 1, '2025-01-10 10:00:00'),
(1, 2, '2025-01-11 12:00:00'),
(2, 1, '2025-01-15 09:00:00'),
(2, 2, '2025-01-16 14:00:00'),
(3, 1, '2025-02-05 11:30:00'),
(3, 3, '2025-02-06 16:00:00');
```

2. Заявка, в която се демонстрира SELECT с ограничаващо условие – ще изведе всички продукти с настояща цена над 2000

```
SELECT * FROM products WHERE current_price > 2000;
```

	id	supplier_id	category_id	name	description	current_price	stock_quantity
►	1	1	1	Dell XPS 15	High-end ultrabook laptop	3200.00	15
	2	1	1	Lenovo ThinkPad X1	Business class laptop	2800.00	20
	3	2	2	iPhone 14 Pro	Apple smartphone	2200.00	30
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

3. Заявка, в която се използва агрегатна функция и GROUP BY – ще изведе общото продадено количество от всеки продукт

```
SELECT products.name AS ProductName,
SUM(order_items.quantity) AS TotalSold
FROM order_items
JOIN products ON order_items.product_id = products.id
GROUP BY products.name;
```

	ProductName	TotalSold
►	Dell XPS 15	2
	Logitech MX Master 3	3
	Lenovo ThinkPad X1	1
	Keychron K6	2
	iPhone 14 Pro	2
	Samsung Galaxy S23	1

4. Заявка, в която се демонстрира INNER JOIN – ще изведе кой дилър какво е поръчал, от кой доставчик е продуктът, какво количество е поръчано и колко ще струва всичко в поръчката

```
SELECT
    orders.id AS OrderId,
    suppliers.name AS SupplierName,
    dealers.name AS DealerName,
    products.name AS ProductName,
    order_items.quantity AS Quantity,
    order_items.unit_price AS ProductPrice,
    (order_items.quantity * order_items.unit_price)
AS TotalPrice,
    orders.order_date AS OrderDate
FROM orders
JOIN dealers ON orders.dealer_id = dealers.id
JOIN order_items ON orders.id = order_items.order_id
JOIN products ON order_items.product_id = products.id
JOIN suppliers ON products.supplier_id = suppliers.id
ORDER BY orders.id;
```

	OrderId	SupplierName	DealerName	ProductName	Quantity	ProductPrice	TotalPrice	OrderDate
►	1	Tech Solutions Ltd.	Ivan Petrov Ltd.	Dell XPS 15	2	3200.00	6400.00	2025-01-10
	1	ElectroWorld	Ivan Petrov Ltd.	Logitech MX Master 3	3	120.00	360.00	2025-01-10
	2	Tech Solutions Ltd.	Ivan Petrov Ltd.	Lenovo ThinkPad X1	1	2800.00	2800.00	2025-01-15
	2	ElectroWorld	Ivan Petrov Ltd.	Keychron K6	2	200.00	400.00	2025-01-15
	3	Global Devices Inc.	Plamen Georgiev EOOD	iPhone 14 Pro	2	2200.00	4400.00	2025-02-05
	4	Global Devices Inc.	Nikola Dimitrov Trade	Samsung Galaxy S23	1	1800.00	1800.00	2025-03-12

5. Заявка, в която се демонстрира OUTER JOIN – ще изведе всички продукти и техния рейтинг, ако има такъв

```
SELECT products.name AS ProductName,
product_ratings.rating AS ProductRating
FROM products
LEFT JOIN product_ratings ON products.id =
product_ratings.product_id;
```

	ProductName	ProductRating
►	Dell XPS 15	5
	Dell XPS 15	4
	Lenovo ThinkPad X1	NULL
	iPhone 14 Pro	4
	Samsung Galaxy S23	5
	Logitech MX Master 3	3
	Keychron K6	5

6. Заявка, в която се демонстрира вложен SELECT – ще изведе всички продукти с цена над средната

```
SELECT products.name AS ProductName,
products.current_price AS ProductPrice
FROM products
WHERE current_price > (
    SELECT AVG(products.current_price)
    FROM products
);
```

	ProductName	ProductPrice
►	Dell XPS 15	3200.00
	Lenovo ThinkPad X1	2800.00
	iPhone 14 Pro	2200.00
	Samsung Galaxy S23	1800.00

7. Заявка, в която се демонстрират едновременно JOIN и агрегатна функция – ще изведе клиентът, който е похарчил най-МНОГО

```
SELECT
    dealers.name AS DealersName,
    SUM(order_items.quantity *
order_items.unit_price) AS TotalSpent
FROM dealers
JOIN orders ON dealers.id = orders.dealer_id
JOIN order_items ON orders.id = order_items.order_id
GROUP BY dealers.id, dealers.name
ORDER BY TotalSpent DESC
LIMIT 1;
```

	DealersName	TotalSpent
►	Ivan Petrov Ltd.	9960.00

8. Тригер – ще запази новия статус на поръчка, след като е променен

```
DELIMITER $
CREATE TRIGGER order_status_update
AFTER UPDATE ON orders
```

```

FOR EACH ROW
BEGIN
    IF OLD.status_id <> NEW.status_id THEN
        INSERT INTO order_status_history(order_id,
status_id, changed_at)
            VALUES (NEW.id, NEW.status_id, NOW());
    END IF;
END $
DELIMITER ;

```

Тестване на тригера:

```

UPDATE orders
SET status_id = 2
WHERE id = 1;

```

```

UPDATE orders
SET status_id = 3
WHERE id = 1;

```

```

SELECT * FROM order_status_history;

```

	id	order_id	status_id	changed_at
	1	1	1	2025-01-10 10:00:00
	2	1	2	2025-01-11 12:00:00
	3	2	1	2025-01-15 09:00:00
	4	2	2	2025-01-16 14:00:00
	5	3	1	2025-02-05 11:30:00
	6	3	3	2025-02-06 16:00:00
	7	1	2	2026-04-27 01:05:43
	8	1	3	2026-04-27 01:05:49

9. Процедура, в която се демонстрира използване на курсор - ще изведе стойността на инвентара на всеки продукт

```

DELIMITER $
CREATE PROCEDURE ProductStockReport()
BEGIN
    DECLARE done INT DEFAULT 0;
    DECLARE product_name VARCHAR(150);
    DECLARE product_stock INT;
    DECLARE product_price DECIMAL(10,2);

    DECLARE product_cursor CURSOR FOR
        SELECT products.name,
products.stock_quantity, products.current_price
        FROM products;

```

```

DECLARE CONTINUE HANDLER FOR NOT FOUND SET done =
1;

CREATE TEMPORARY TABLE temp_report (
    product VARCHAR(150),
    price DECIMAL(10,2),
    stock INT,
    inventory_value DECIMAL(10,2)
);

OPEN product_cursor;
read_loop: LOOP
    FETCH product_cursor INTO product_name,
product_stock, product_price;

    IF done THEN
        LEAVE read_loop;
    END IF;

    INSERT INTO temp_report
VALUES (
        product_name,
        product_price,
        product_stock,
        product_stock * product_price
    );
END LOOP;
CLOSE product_cursor;

SELECT * FROM temp_report;

DROP TEMPORARY TABLE temp_report;
END $
DELIMITER ;

```

	product	price	stock	inventory_value
►	Dell XPS 15	3200.00	15	48000.00
	Lenovo ThinkPad X1	2800.00	20	56000.00
	iPhone 14 Pro	2200.00	30	66000.00
	Samsung Galaxy S23	1800.00	25	45000.00
	Logitech MX Master 3	120.00	100	12000.00
	Keychron K6	200.00	60	12000.00